



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

NOMBRE DEL TRABAJO:

Reporte Python

PRESENTA:

Pedro Gopar Merino 20350643

GRADO Y GRUPO: 8 A

DOCENTE:

JULIO AGUILAR CARMONA

CARRERA:



DEPARTAMENTO:
SISTEMAS Y COMPUTACIÓN

TecNM · Ingeniería en Sistemas Computacionales

Ciencia de Datos

Documentación Técnica de Prácticas 2 – 7

Análisis Industrial · Logística · Reducción de Dimensionalidad · Machine Learning

Librerías: NumPy · Pandas · Matplotlib · Seaborn · Scikit-learn · SciPy

2025

Tabla de Contenidos

Tabla de Contenidos	1
Práctica 2: Análisis de Calidad Industrial — TechManufacture	3
Sección 0 — Importación de Librerías.....	3
Sección 1 — Generación Sintética del Dataset	3
Fase A — Análisis Exploratorio (EDA).....	4
A.1 · Estadísticas Descriptivas	4
A.2 · Valores Faltantes e Imputación.....	4
A.3 · Correlación de Pearson	4
Fase B — Agrupación y Segmentación	4
Fase C — Visualización (4 Figuras)	5
Práctica 3: Análisis de Eficiencia en Logística Global — Global-Logistics ISC.....	8
Sección 1 — Generación del Dataset (600 envíos).....	8
Fase 1 — Calidad de Datos.....	8
Fase 2 — Correlación de Pearson	8
Fase 3 — Comparativa de Modalidades.....	9
Fase 4 — Visualización (4 Figuras)	9
Práctica 4: Reducción de Dimensionalidad — Sensores Industriales Metal-Tech	12

Fase 1 — Generación del Dataset (12 Sensores, 200 lecturas)	13
Fase 2 — Estandarización y PCA	13
Fase 3 — Scree Plot y Selección de Componentes	13
Fase 3 (cont.) — Cargas (Loadings)	14
Fase 4 — Biplot	14
Práctica 5: Reducción de Dimensiones en Tráfico de Red — Cyber-Systems	17
Paso 1 — Dataset con Dependencias de Red	17
Paso 2 — Estandarización y PCA	17
Paso 3 — Análisis de Varianza con Doble Umbral	17
Paso 4 — Loadings: De Métricas a Conceptos	17
Paso 5 — Biplot: Mapa de Estado de Red	18
Práctica 6: Optimización de Sensores en Smart Cities — LoRaWAN	19
Paso 1 — Dataset con Lógica de Ciudad	20
Paso 2 — Estandarización	20
Paso 3 — Varianza y Ahorro de Transmisión	20
Paso 4 — Heatmap de Loadings	20
Paso 5 — Biplot Coloreado por Grupo Temático	21
Práctica 7: Machine Learning — Regresión, Clasificación y Clustering (RRHH)	23
Paso 1 — Dataset de Empleados con Sigmoid	24
Paso 2 — Regresión Lineal con Estadísticos t	24
Paso 3 — Clasificación KNN con GridSearchCV	25
Paso 4 — Clustering K-Means	25
Resumen General: Librerías y Herramientas por Práctica	28

Práctica 2: Análisis de Calidad Industrial — TechManufacture

Archivo: practica2.py

Objetivo: Analizar 500 lotes de producción para identificar la relación entre temperatura y presión de operación con la tasa de error del producto final.

Sección 0 — Importación de Librerías

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
matplotlib.use("TkAgg") # backend para ventana nativa
```

Librería	Uso en la práctica
numpy	Generación de datos, operaciones vectoriales
pandas	DataFrame de lotes, agrupaciones, estadísticas
matplotlib	Cuatro figuras: dashboard, boxplot, scatter, KDE
seaborn	histplot, violinplot, boxplot, kdeplot, heatmap
scipy.stats	pearsonr (correlación), ttest_ind (Welch t-test)
sklearn	LinearRegression para regresión simple, r2_score

El bloque `plt.rcParams` configura un tema visual oscuro (fondo #0d1117) que imita el estilo GitHub Dark, mejorando el contraste para presentaciones industriales. `matplotlib.use("TkAgg")` fuerza el backend nativo del sistema operativo para que los gráficos se abran en ventanas interactivas.

Sección 1 — Generación Sintética del Dataset

```
np.random.seed(42)
N = 500
turno = np.where(np.arange(N) % 2 == 0, "Matutino", "Vespertino")
temperatura_base = np.where(turno == "Matutino",
                             np.random.normal(72, 6, N),
                             np.random.normal(76, 7, N))
presion = 4.5 + 0.03 * temperatura_base + np.random.normal(0, 0.4, N)
tasa_error = (-2.5 + 0.08*temperatura_base
              + 0.004*temperatura_base**2
              + np.random.normal(0, 1.2, N)).clip(0)
```

- `np.random.seed(42)`: fija la semilla para reproducibilidad total.
- `np.where(arange % 2 == 0, ...)`: asigna turnos alternados de forma determinista — los lotes pares son Matutinos.

- Turno Vespertino opera con temperatura media 4 °C más alta (76 vs 72): simula degradación térmica acumulada a lo largo del día.
- $\text{presion} = 4.5 + 0.03 \cdot \text{temperatura}$: introduce una correlación positiva débil entre temperatura y presión.
- `tasa_error` sigue un modelo cuadrático-lineal: a temperaturas extremas, el error escala más rápido. `clip(0)` evita tasas de error negativas físicamente imposibles.
- 4 % de NA: `np.random.choice` introduce valores faltantes en temperatura para simular fallos de sensor.

Fase A — Análisis Exploratorio (EDA)

A.1 · Estadísticas Descriptivas

```
media_temp = df["temperatura"].mean()
mediana_temp = df["temperatura"].median()
std_temp = df["temperatura"].std(ddof=1) # estimador insesgado
```

- `ddof=1` usa el denominador N-1 (corrección de Bessel): produce el estimador insesgado de la desviación estándar poblacional.
- Coeficiente de Variación (CV) = $s/\mu \times 100$: mide la variabilidad relativa. CV < 15 % = proceso estable; > 15 % = alta variabilidad.

A.2 · Valores Faltantes e Imputación

```
df["temperatura"] = df["temperatura"].fillna(df["temperatura"].median())
```

- Se usa la mediana en lugar de la media porque es robusta ante valores atípicos (outliers).
- Si hubiera lotes con temperatura extrema (outlier), la media se vería sesgada y generaría una imputación incorrecta; la mediana no.

A.3 · Correlación de Pearson

```
r_pearson, p_valor = stats.pearsonr(df["temperatura"], df["tasa_error"])
```

- `scipy.stats.pearsonr` devuelve: (1) coeficiente $r \in [-1, 1]$ y (2) valor-p para $H_0: r = 0$.
- Si $p < 0.05$ se rechaza H_0 y se concluye que la correlación es estadísticamente significativa.
- $r > 0$ indica correlación positiva: a mayor temperatura, mayor tasa de error.

Fase B — Agrupación y Segmentación

```
resumen_turno = df.groupby("turno").agg(
    n=("tasa_error", "count"), media_err=("tasa_error", "mean"), ...)

t_stat_err, p_err = stats.ttest_ind(mat, ves, equal_var=False) # Welch
```

- `groupby("turno").agg()` calcula simultáneamente *n*, media, desviación estándar y mediana por turno.
- Welch t-test (`equal_var=False`): prueba de hipótesis que NO asume varianzas iguales entre grupos — más robusta que el t-test clásico de Student.
- $H_0: \mu_{\text{Matutino}} = \mu_{\text{Vespertino}}$. Si $p < 0.05 \rightarrow$ los turnos tienen tasas de error significativamente distintas.
- Se aplica también a la variable temperatura para confirmar si la diferencia térmica entre turnos es real o aleatoria.

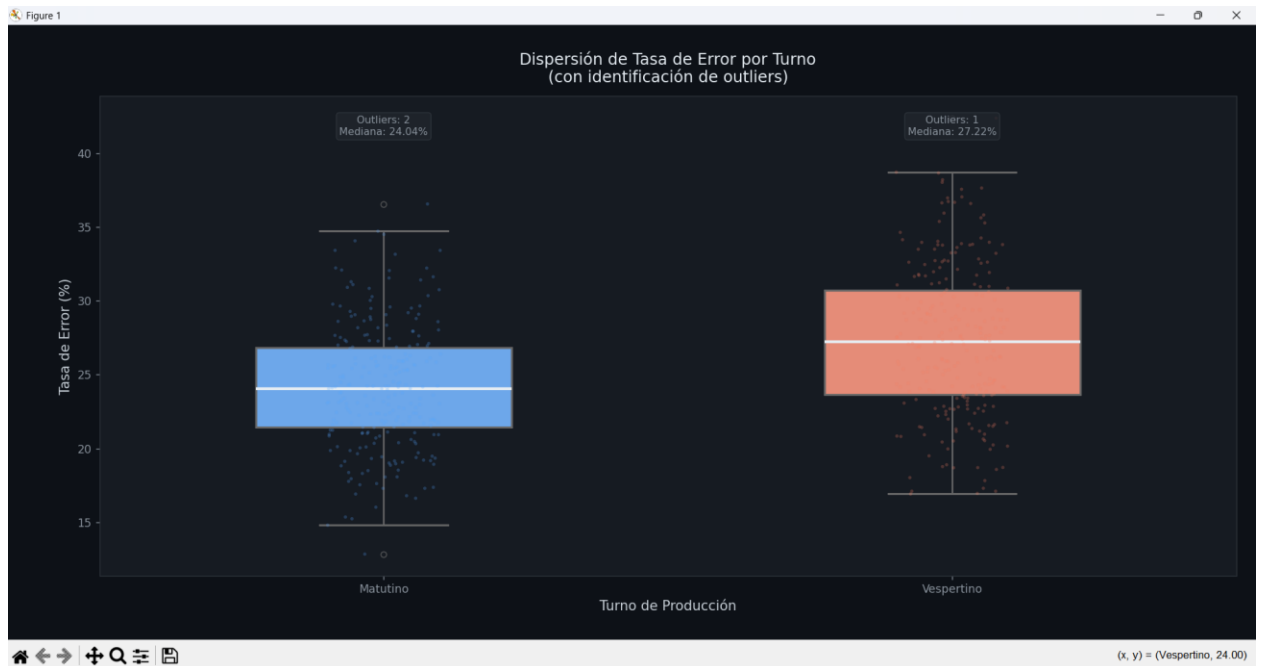
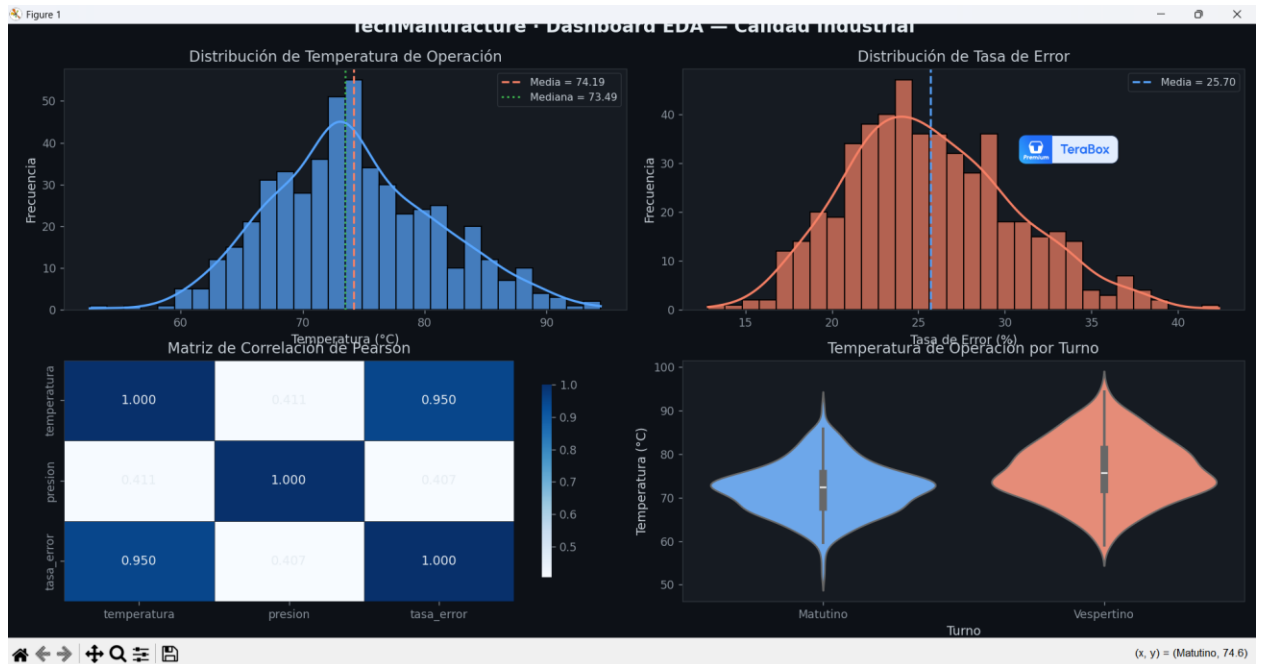
Fase C — Visualización (4 Figuras)

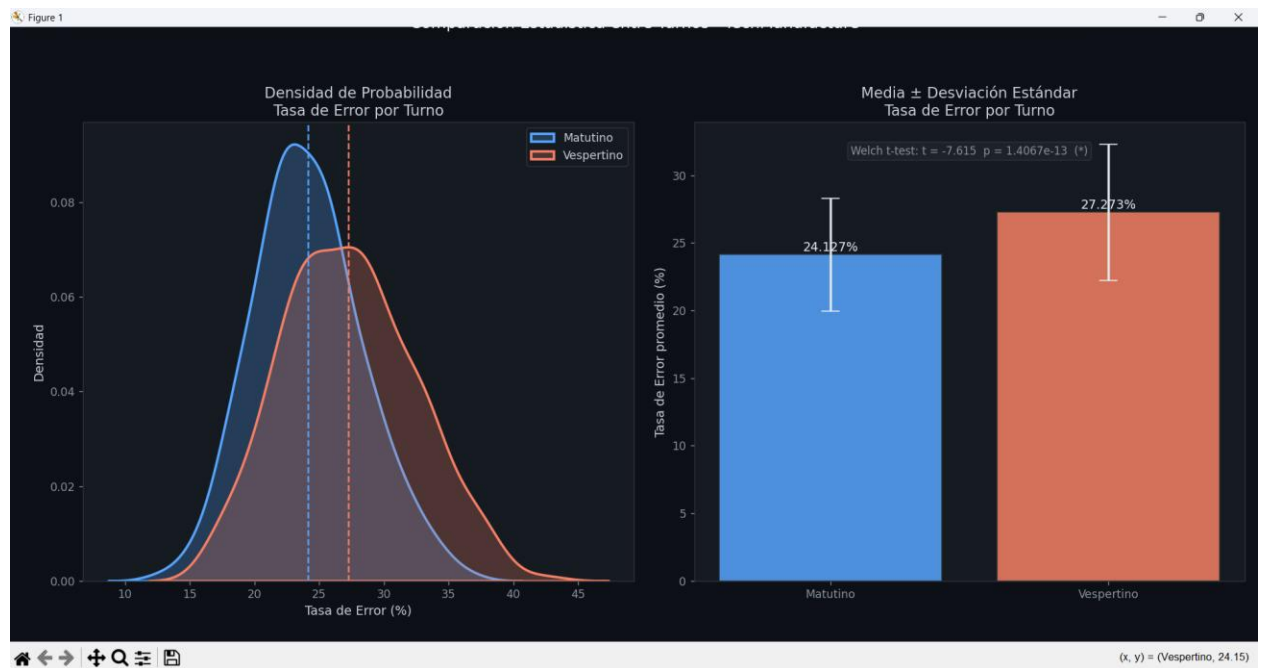
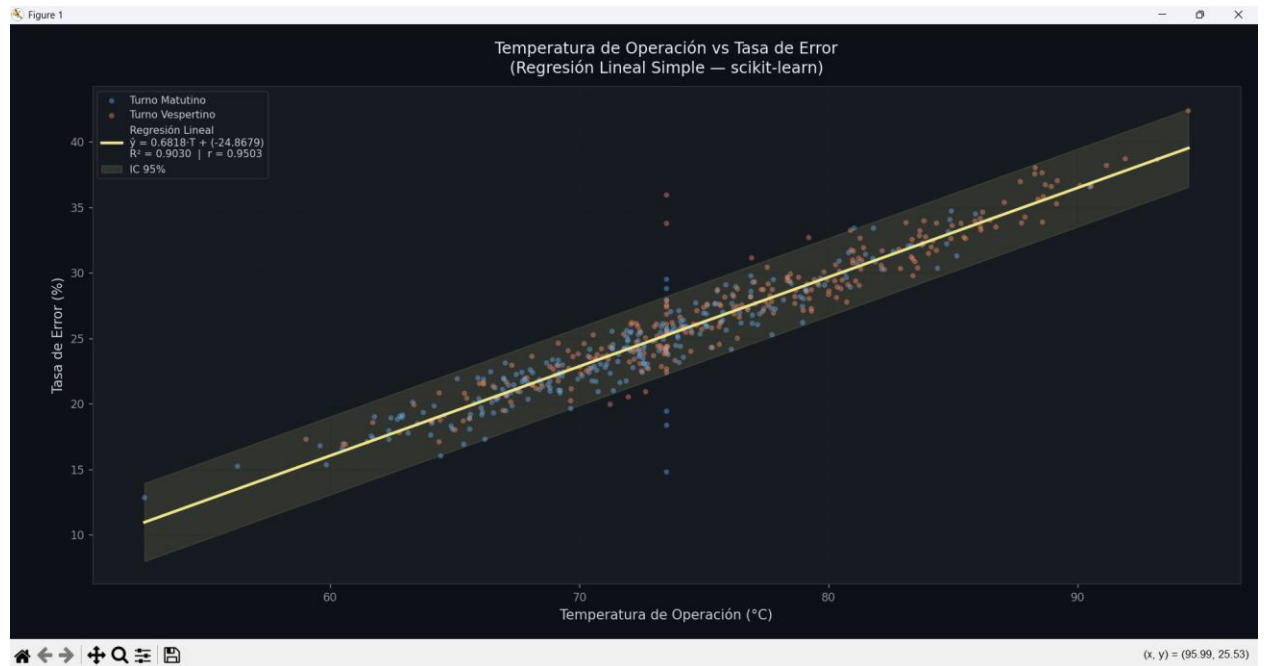
Figura	Tipo de gráfica	Qué muestra
Fig. 1 (2×2)	Histograma+KDE, Heatmap, Violin	Dashboard EDA: distribuciones y correlaciones
Fig. 2	Boxplot + Stripplot	Dispersión de <code>tasa_error</code> por turno; outliers detectados con IQR
Fig. 3	Scatter + Regresión + IC 95%	Temperatura vs <code>tasa_error</code> : línea de regresión y banda de confianza
Fig. 4 (1×2)	KDE + Barras ± DE	Densidad y media por turno; resultado del t-test anotado

```
# Regresión lineal con scikit-learn (Figura 3)
X = df["temperatura"].values.reshape(-1, 1)
modelo = LinearRegression()
modelo.fit(X, y)
r2 = r2_score(y, modelo.predict(X))

# Intervalo de confianza visual ±1.96·SE
residuos = y - modelo.predict(X)
se = np.std(residuos, ddof=2)
ax.fill_between(x_range, y_range - 1.96*se, y_range + 1.96*se, alpha=0.12)
```

- `reshape(-1, 1)`: sklearn exige que *X* sea una matriz 2D, no un vector. El -1 indica "calcular automáticamente esa dimensión".
- R^2 mide qué fracción de la varianza de `tasa_error` es explicada por temperatura.
- IC 95% visual: $\pm 1.96 \cdot SE_{\text{residuos}}$. Aproximación válida para *n* grande; muestra el rango esperado de predicciones.
- La banda IC se dibuja con `fill_between`, un área sombreada entre dos curvas.





Práctica 3: Análisis de Eficiencia en Logística Global — Global-Logistics ISC

Archivo: practica3.py

Objetivo: Analizar 600 envíos para identificar cuellos de botella en las modalidades Terrestre y Aéreo bajo distintas condiciones climáticas.

Sección 1 — Generación del Dataset (600 envíos)

```
np.random.seed(7)
N = 600
tipo_transporte = np.where(np.arange(N) % 2 == 0, "Terrestre", "Aéreo")
condicion_clima = np.random.choice(["Normal", "Lluvia", "Tormenta"],
                                   size=N, p=[0.60, 0.28, 0.12])

velocidad = np.where(tipo_transporte == "Terrestre", 65, 480)
tiempo_entrega_hrs = (distancia_base / velocidad
                      + penalizacion_tiempo
                      + np.random.normal(0, 1.5, N)).clip(0.5)
```

- `condicion_clima` con probabilidades [0.60, 0.28, 0.12]: simula frecuencias reales de clima — 60 % Normal, 28 % Lluvia, 12 % Tormenta.
- `velocidad` efectiva: 65 km/h Terrestre vs 480 km/h Aéreo. El tiempo base = distancia / velocidad.
- `penalizacion_tiempo`: +3 hrs por Lluvia, +8 hrs por Tormenta — refleja el impacto climático en los tiempos reales de entrega.
- `clip(0.5)`: ningún envío puede entregarse en menos de 30 minutos (mínimo físico razonable).
- 5 % de NA en `distancia_km` simula fallos del GPS a bordo del vehículo.

Fase 1 — Calidad de Datos

```
mediana_dist = df["distancia_km"].median()
df["distancia_km"] = df["distancia_km"].fillna(mediana_dist)
```

- Igual que en práctica 2: imputación por mediana por su robustez ante outliers de distancia.
- `df.describe()` imprime estadísticos de todas las columnas numéricas en una sola llamada.

Fase 2 — Correlación de Pearson

```
r_pearson, p_pearson = stats.pearsonr(df["distancia_km"],
                                       df["tiempo_entrega_hrs"])

fuerza = ("muy fuerte" if abs(r) >= 0.8 else
          "fuerte"     if abs(r) >= 0.6 else
          "moderada"   if abs(r) >= 0.4 else "débil")
```

- La correlación global distancia↔tiempo es moderada porque la modalidad (Terrestre vs Aéreo) y el clima introducen varianza adicional que "rompe" la linealidad perfecta.
- El código clasifica automáticamente la fuerza de la correlación con umbrales estándar (Cohen 1988).

Fase 3 — Comparativa de Modalidades

```
terrestre = df.loc[df["tipo_transporte"] == "Terrestre", "tiempo_entrega_hrs"]
aereo     = df.loc[df["tipo_transporte"] == "Aéreo",      "tiempo_entrega_hrs"]

t_stat, p_val = stats.ttest_ind(terrestre, aereo, equal_var=False)

# Tamaño del efecto
pooled_std = np.sqrt((terrestre.std(ddof=1)**2 + aereo.std(ddof=1)**2) / 2)
cohen_d     = (terrestre.mean() - aereo.mean()) / pooled_std
```

- Welch t-test: evalúa si la diferencia de medias entre Terrestre y Aéreo es estadísticamente real o podría deberse al azar.
- Cohen's d = diferencia de medias / desviación estándar agrupada. Interpreta el tamaño práctico del efecto:
 - $|d| \geq 0.8 \rightarrow$ efecto grande (diferencia clínicamente importante)
 - $|d| \in [0.5, 0.8) \rightarrow$ efecto mediano
 - $|d| < 0.5 \rightarrow$ efecto pequeño (diferencia estadística pero no práctica)

Fase 4 — Visualización (4 Figuras)

Figura	Tipo	Contenido
Fig. 1 (2×2)	Histograma, Heatmap, Violin	Dashboard EDA: tiempo, distancia por modalidad, costo
Fig. 2	Scatter + 3 regresiones + IC	Distancia vs tiempo: curva global + curvas por modalidad
Fig. 3	Boxplot + Stripplot + barra p	Tiempo por modalidad con p-value del t-test anotado
Fig. 4 (1×2)	KDE + Barras agrupadas	Densidad por modalidad; tiempo promedio por clima × modalidad

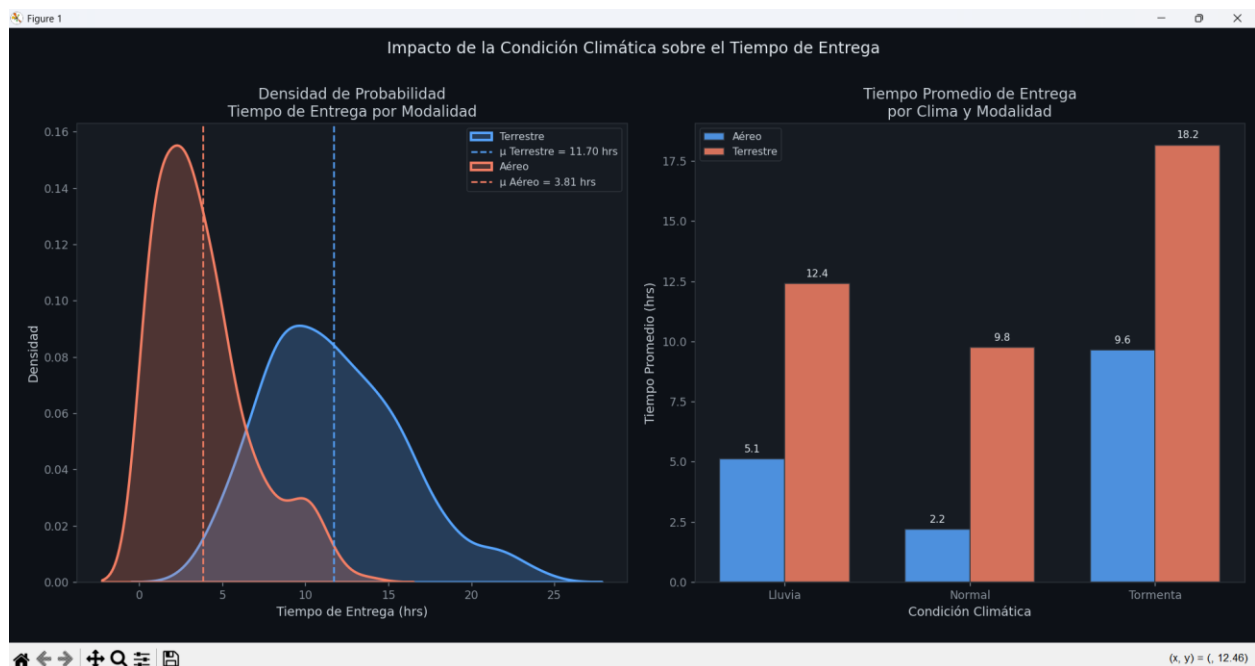
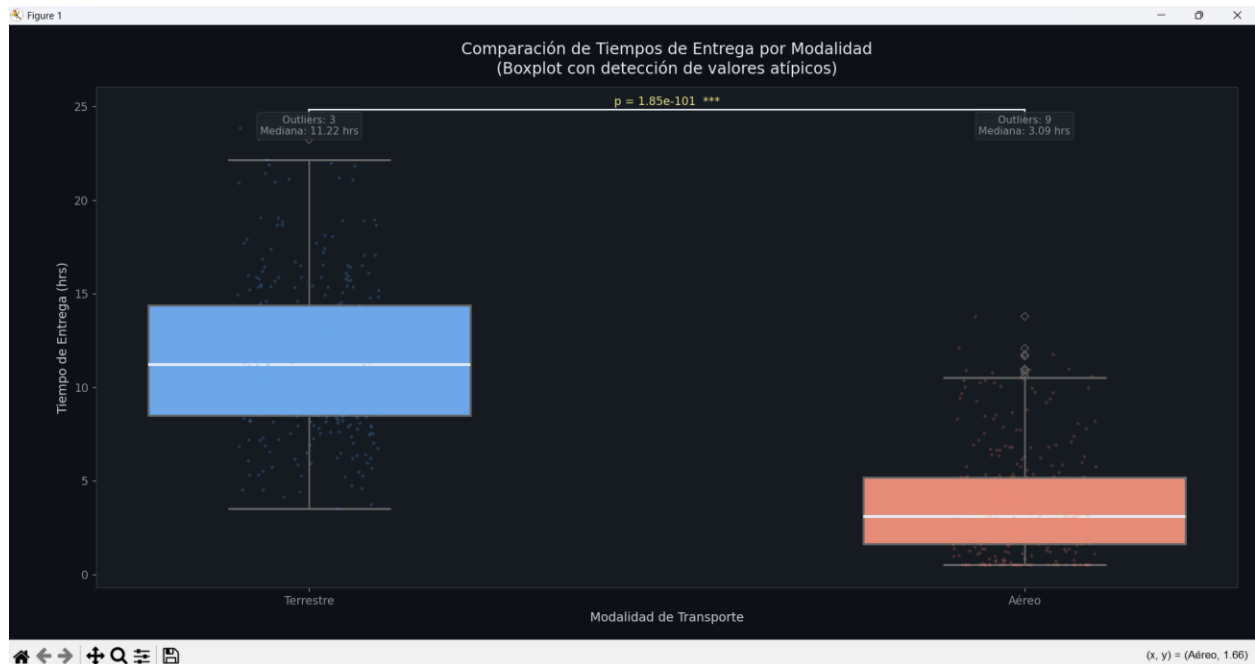
```
# Regresiones separadas por modalidad (Figura 2)
for mod, color in PALETTE.items():
    Xm = sub["distancia_km"].values.reshape(-1, 1)
    m = LinearRegression().fit(Xm, ym)
    ax2.plot(xr, m.predict(xr), linestyle="--", label=f"Regresión {mod}")
```

- Se ajustan tres regresiones en la misma figura: global (línea sólida) y una por modalidad (líneas discontinuas).
- Las pendientes distintas revelan que la misma distancia adicional tiene diferente impacto en tiempo según la modalidad.

```
# Barra de significancia entre grupos (Figura 3)
y_bar = df["tiempo_entrega_hrs"].max() * 1.04
ax3.plot([0, 1], [y_bar, y_bar], color="#e6edf3", lw=1.4)
sig_label = f"p = {p_val:.2e}  {'***' if p_val<0.001 else ...}"
ax3.text(0.5, y_bar*1.005, sig_label, ha="center")
```

- La barra horizontal con el valor-p es la notación estándar en publicaciones científicas para indicar comparaciones estadísticas entre grupos.





Práctica 4: Reducción de Dimensionalidad — Sensores Industriales Metal-Tech

Archivo: practica4.py

Objetivo: Aplicar PCA sobre 12 sensores de una caldera para reducir dimensiones al mínimo de componentes que expliquen $\geq 85\%$ de la varianza, aliviando la carga del PLC.

Fase 1 — Generación del Dataset (12 Sensores, 200 lecturas)

```
np.random.seed(42)
n = 200
temp1 = np.random.normal(100, 5, n)
temp2 = temp1 + np.random.normal(0, 1, n) # correlación ~0.98 con temp1
vibracion = (temp1 * 0.5) + np.random.normal(0, 2, n) # correlación ~0.97 con temp1
co2 = temp1 * 0.2 + np.random.normal(0, 0.5, n) # correlación alta
ruido_db = vibracion * 1.2 + np.random.normal(0, 1, n)
eficiencia = 100 - (temp1 * 0.1)
```

- temp2 ≈ temp1: simula dos sensores de temperatura casi redundantes (escape ≈ núcleo + pequeño ruido).
- vibracion, ruido_db y eficiencia son derivadas lineales de temp1: crean un grupo de variables altamente correlacionadas que el PCA colapsará en un solo componente.
- humedad y voltaje son independientes: no se correlacionan con las demás → el PCA las asignará a componentes propios.

Fase 2 — Estandarización y PCA

```
scaler = StandardScaler()
datos_escalados = scaler.fit_transform(datos_sensores)

pca = PCA() # calcula todos los componentes
pca.fit(datos_escalados)

var_prop = pca.explained_variance_ratio_
var_acum = np.cumsum(var_prop)
```

- StandardScaler: transforma cada columna a media = 0, desviación estándar = 1. Obligatorio en PCA para que ninguna variable domine por tener mayor escala numérica.
- fit_transform = fit() + transform() en un solo paso: ajusta los parámetros (media y SD) y aplica la transformación.
- pca.explained_variance_ratio_: fracción de varianza total explicada por cada PC. PC1 siempre es el mayor.
- np.cumsum(): suma acumulada progresiva → permite encontrar cuántos PCs se necesitan para el umbral deseado.

Fase 3 — Scree Plot y Selección de Componentes

```
n_85 = int(np.argmax(var_acum >= 0.85)) + 1

# Scree Plot
axes[0].axhline(y=1/n_componentes, color="red", linestyle="--",
                label="Criterio Kaiser (1/p)")
```

```
# Varianza acumulada con umbral
axes[1].axhline(y=85, color="green", linestyle="--", label="Umbral 85%")
axes[1].axvline(x=n_85, color="purple", linestyle=":")
```

- `np.argmax(condición) + 1`: encuentra el índice del primer valor que cumple la condición. +1 porque los índices son base-0 pero los PCs son base-1.
- Criterio Kaiser: retener PCs cuya varianza > 1/p (promedio). Línea roja horizontal en el Scree Plot.
- El "codo" es el punto donde la curva del Scree Plot se aplana: agregar más PCs después del codo aporta poco.
- Línea verde (85%) y morada (PC óptimo) en la gráfica de varianza acumulada guían la decisión visual.

Fase 3 (cont.) — Cargas (Loadings)

```
loadings = pd.DataFrame(
    pca.components_[:2].T, # forma: (n_vars, 2)
    index=datos_sensores.columns,
    columns=["PC1", "PC2"]
)
print(loadings["PC1"][loadings["PC1"].abs() > 0.3]...)
```

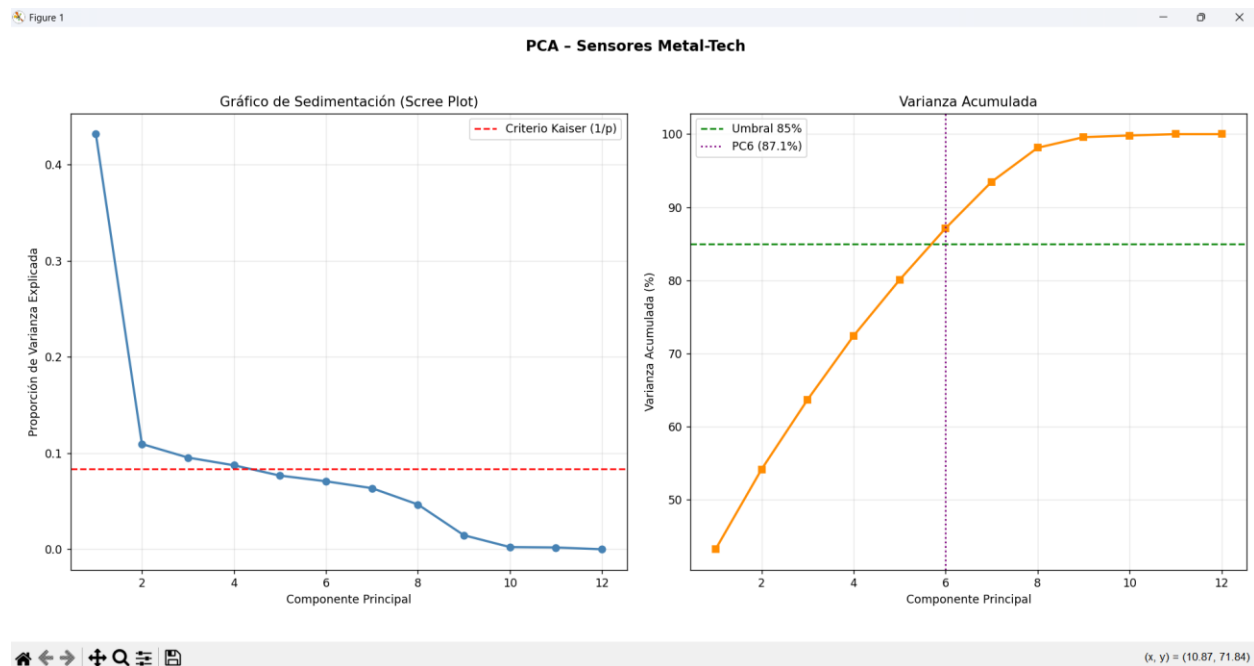
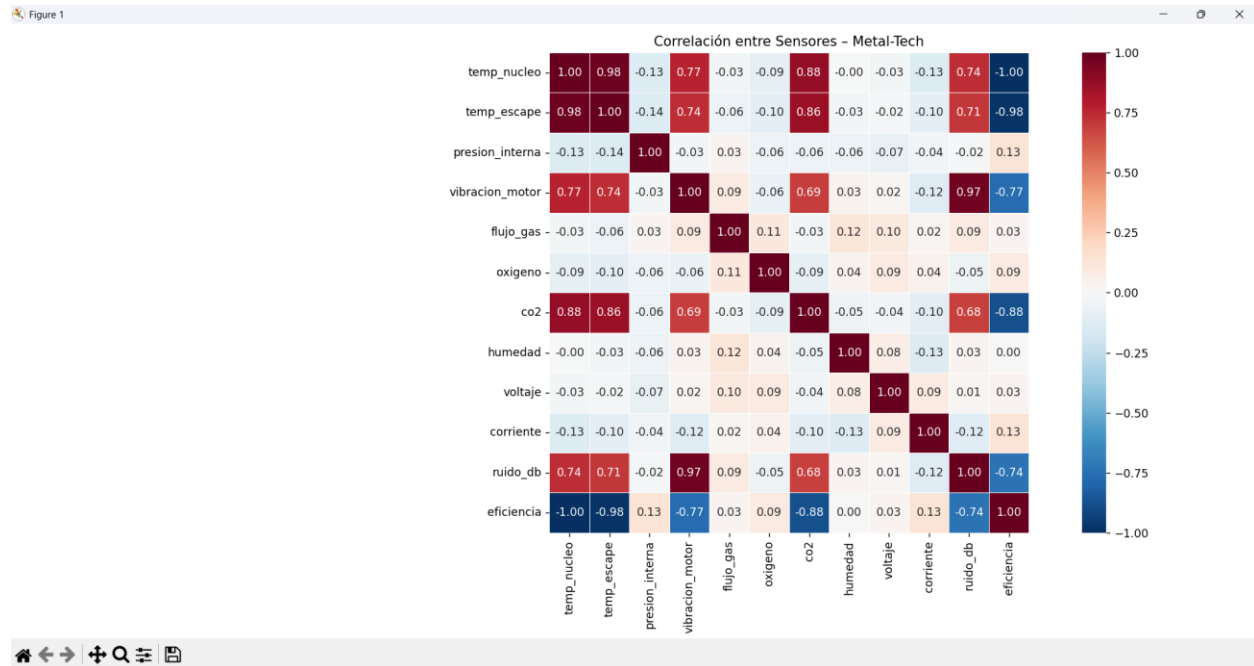
- `pca.components_`: matriz de forma (n_pcs, n_variables). Cada fila es un PC; cada columna es la carga de esa variable en ese PC.
- `[:2].T` transpone para obtener (n_variables, 2): tabla legible de sensor × PC.
- Carga alta en PC1 ($|carga| > 0.3$): esa variable "apunta" en la dirección de mayor varianza. Indica qué sensores definen PC1.
- Interpretación: si `temp_nucleo`, `co2`, `ruido_db`, `eficiencia` tienen cargas altas en PC1 → PC1 = "Estado Térmico de la Caldera".

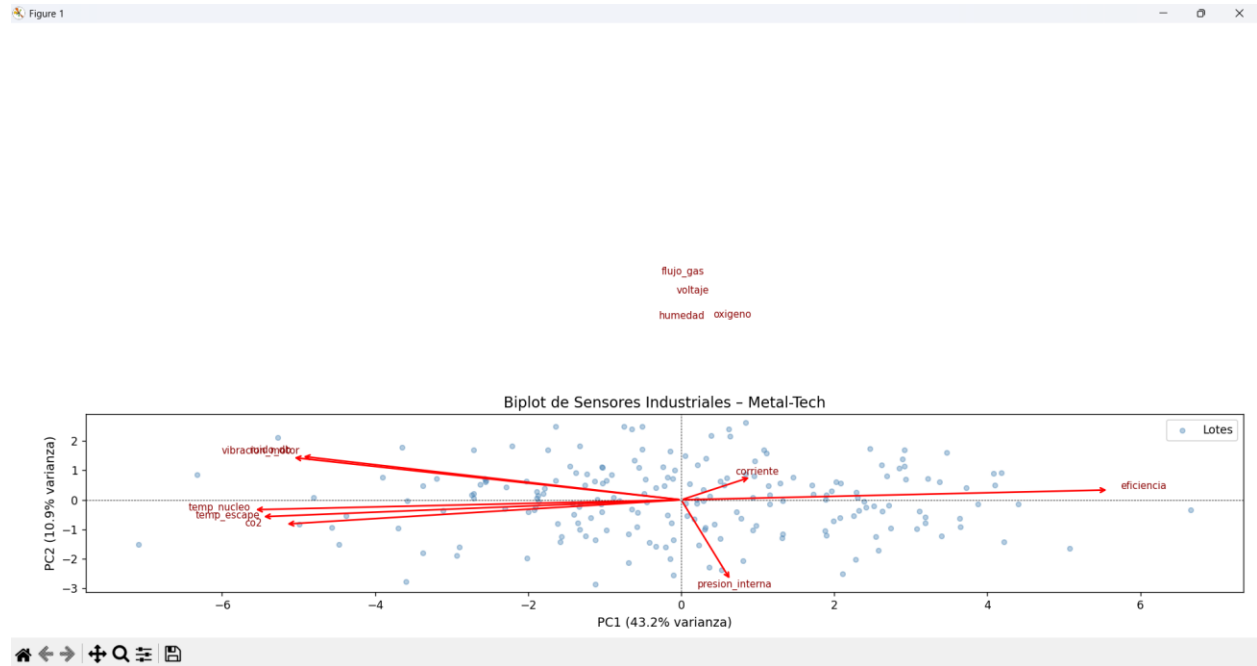
Fase 4 — Biplot

```
scores = pca.transform(datos_escalados) # puntos observados en espacio PC
scale = np.max(np.abs(scores[:, :2])) / np.max(np.abs(comps[:2]))

for i, var in enumerate(datos_sensores.columns):
    ax.annotate("", xy=(comps[0,i]*scale, comps[1,i]*scale), xytext=(0,0),
        arrowprops=dict(arrowstyle="->", color="red", lw=1.5))
```

- `scores`: cada fila es una lectura (lote) proyectada al espacio PC1–PC2.
- `scale`: factor que ajusta las flechas para que sean visibles en el mismo espacio que los puntos.
- Flechas paralelas → variables redundantes. Flechas perpendiculares → variables independientes. Flechas opuestas → correlación negativa.
- Puntos alejados del centro → lecturas atípicas de la caldera (posible fallo incipiente).





Práctica 5: Reducción de Dimensiones en Tráfico de Red — Cyber-Systems

Archivo: practica5.py

Objetivo: Reducir 10 métricas de conexión de red a los componentes principales que distingan el estado Normal del Anómalo, aliviando el procesamiento en tiempo real.

Paso 1 — Dataset con Dependencias de Red

```
datos_red["bytes_enviados"] = datos_red["paquetes_enviados"] * 1500
                             + np.random.normal(0, 500, n)
datos_red["reintentos_tcp"] = datos_red["errores_checksum"] * 1.5
                             + np.random.normal(0, 0.5, n)
datos_red["carga_cpu_router"] = paquetes * 0.4 + latencia * 0.2
```

- bytes_enviados $\approx 1500 \times$ paquetes_enviados: el MTU (Maximum Transmission Unit) de Ethernet es 1500 bytes $\rightarrow$ relación lineal casi perfecta.
- reintentos_tcp $\uparrow$ cuando errores_checksum $\uparrow$: TCP retransmite automáticamente paquetes con checksum incorrecto.
- carga_cpu_router depende del volumen y la latencia: a más paquetes + mayor latencia, más CPU consume el router procesando colas.
- Estas 3 dependencias crean redundancia intencional que el PCA debe detectar y comprimir.

Paso 2 — Estandarización y PCA

- Misma lógica que práctica 4. bytes_enviados está en decenas de miles; jitter en ~ 2 . Sin escalar, bytes dominaría el PCA.

Paso 3 — Análisis de Varianza con Doble Umbral

```
n_70 = int(np.argmax(var_acum >= 0.70)) + 1
n_85 = int(np.argmax(var_acum >= 0.85)) + 1
print(f"PC1 + PC2 explican: {var_acum[1]*100:.1f}%")

# Scree Plot con dos umbrales
axes[1].axhline(y=70, color="blue", label="Umbral 70%")
axes[1].axhline(y=85, color="green", label="Umbral 85%")
```

- Se evalúan dos umbrales: 70 % (mínimo aceptable) y 85 % (recomendado para monitoreo de red crítica).
- Si PC1 + PC2 > 70 %: se puede visualizar el estado de la red en un plano 2D con aceptable fidelidad.

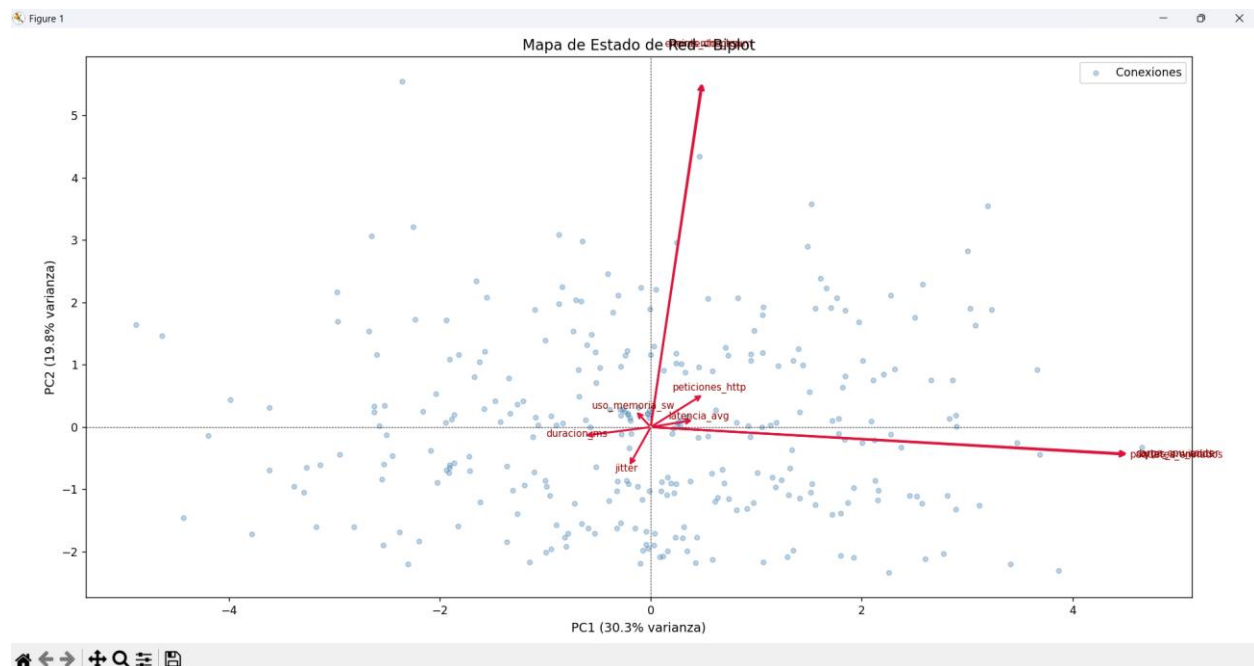
Paso 4 — Loadings: De Métricas a Conceptos

```
top_pc1 = loadings["PC1"][loadings["PC1"].abs() > 0.30]
        .sort_values(key=abs, ascending=False)
```

- PC1 con cargas altas en paquetes_enviados, bytes_enviados, carga_cpu_router → PC1 = "Volumen de Tráfico".
- PC2 con cargas altas en latencia_avg, jitter → PC2 = "Calidad y Estabilidad de la Conexión".
- Las 10 métricas técnicas se reducen a 2 conceptos de ingeniería interpretables por un operador.

Paso 5 — Biplot: Mapa de Estado de Red

- Flechas paralelas (bytes y paquetes): confirma su redundancia — se puede descartar una del bus de datos.
- Puntos aislados en los extremos: conexiones con comportamiento anómalo (posibles ataques DDoS o fallas de switch).



Práctica 6: Optimización de Sensores en Smart Cities — LoRaWAN

Archivo: practica6.py

Objetivo: Reducir 10 sensores urbanos por intersección (calidad de aire, ruido, tráfico, clima) para ahorrar ancho de banda en la red LoRaWAN de bajo consumo.

Paso 1 — Dataset con Lógica de Ciudad

```
np.random.seed(456)
n = 400
co2_ppm = densidad_vehiculos * 3.5 + normal(300, 50)
no2_ppb = co2_ppm * 0.1 + normal(5, 2)
particulas_pm25 = co2_ppm * 0.05 + normal(10, 3)
nivel_ruido_db = densidad_vehiculos * 0.2 + 50 + normal(0, 5)
```

- `co2_ppm` ↑ con `densidad_vehiculos`: más vehículos = más combustión = más CO₂.
- `no2_ppb` deriva del `co2_ppm`: el NO₂ se genera en la misma combustión.
- `particulas_pm25` también provienen de la combustión vehicular (correlación con CO₂).
- `nivel_ruido_db` ↑ con `densidad`: más tráfico = más ruido.
- Resultado: 4 sensores (`co2`, `no2`, `pm25`, `ruido`) miden prácticamente lo mismo → PCA los fusionará.

Paso 2 — Estandarización

Paso 3 — Varianza y Ahorro de Transmisión

```
n_85 = int(np.argmax(var_acum >= 0.85)) + 1
ahorro_pct = (1 - n_85 / datos_urbanos.shape[1]) * 100

# Área rellena bajo la curva hasta el umbral
axes[1].fill_between(range(1, n_85+1), var_acum[:n_85]*100,
                    alpha=0.15, color="green")
```

- `ahorro_pct = (1 – componentes_retenidos / sensores_totales) × 100`: porcentaje de datos que se deja de transmitir sin pérdida crítica de información.
- `fill_between`: rellena el área bajo la curva de varianza acumulada hasta el umbral, visualizando el "espacio capturado".

Paso 4 — Heatmap de Loadings

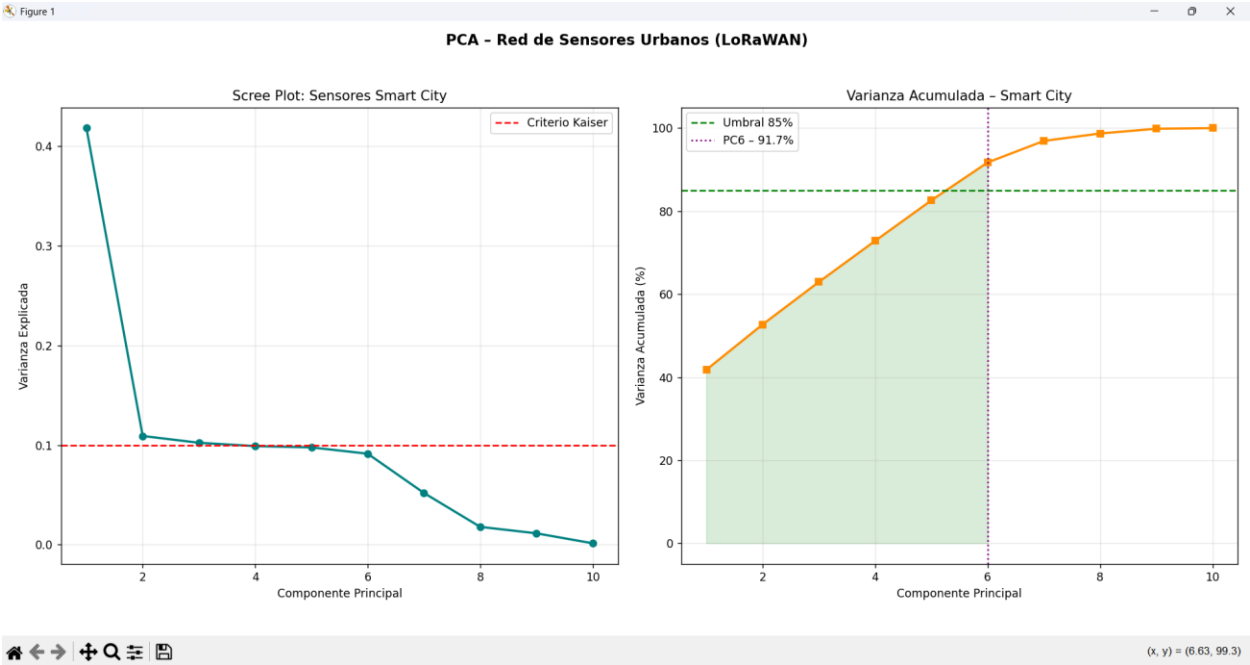
```
sns.heatmap(loadings, annot=True, fmt=".3f",
            cmap="coolwarm", center=0, vmin=-1, vmax=1)
```

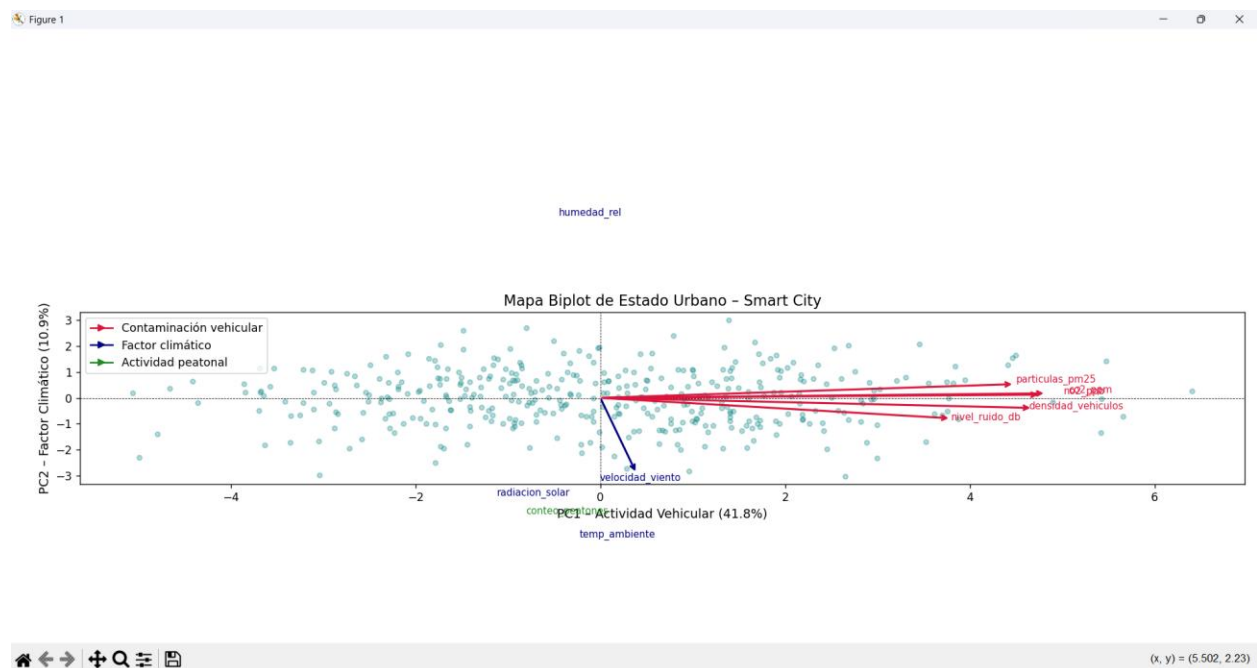
- `coolwarm` con `center=0`: rojo = carga positiva alta, azul = carga negativa alta, blanco = carga nula.
- `vmin=-1, vmax=1`: fija la escala al rango teórico de las cargas (cosenos de ángulos).
- PC1: cargas rojas en `co2`, `no2`, `pm25`, `ruido`, `densidad` → "Actividad e Impacto Vehicular".
- PC2: cargas rojas en `temp_ambiente`, `humedad`, `radiacion_solar` → "Factor Climático/Meteorológico".

Paso 5 — Biplot Coloreado por Grupo Temático

```
colors_var = {
    "co2_ppm": "crimson", "no2_ppb": "crimson",    # contaminación
    "temp_ambiente": "navy", "humedad_rel": "navy", # clima
    "conteo_peatones": "forestgreen"             # actividad
}
from matplotlib.lines import Line2D # leyenda manual
```

- Colorear las flechas por grupo temático facilita la interpretación visual sin necesidad de leer etiquetas.
- Line2D se usa para crear una leyenda manual cuando el color de las flechas no es gestionado por matplotlib automáticamente.
- Flechas rojo paralelas entre sí (co2, no2, pm25): confirman la redundancia → se puede transmitir solo uno y reconstruir los demás.





Práctica 7: Machine Learning — Regresión, Clasificación y Clustering (RRHH)

Archivo: practica7.py

Objetivo: Automatizar el departamento de RRHH de DataSystems S.A. prediciendo salario (regresión), retención (clasificación KNN) y segmentando empleados (K-Means).

Paso 1 — Dataset de Empleados con Sigmoid

```
np.random.seed(789)
n = 500

df_empleados['salario'] = (25000
    + df_empleados['experiencia'] * 5000
    + df_empleados['certificaciones'] * 2000
    + np.random.normal(0, 3000, n))

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

prob = sigmoid(-2 + 0.0001*salario + 0.2*habilidades_sociales)
df_empleados['retencion'] = (np.random.uniform(size=n) < prob).astype(int)
```

- Salario: relación lineal con experiencia (+\$5,000/año) y certificaciones (+\$2,000 c/u) más ruido gaussiano de $\pm \$3,000$.
- $\text{sigmoid}(x) = 1/(1+e^{-x})$: función que convierte cualquier valor real en una probabilidad $\in (0,1)$. Es la inversa del logit.
- $-2 + 0.0001 \cdot \text{salario} + 0.2 \cdot \text{habilidades}$: empleados con mayor salario y mejores habilidades sociales tienen mayor probabilidad de quedarse.
- $(\text{uniform} < \text{prob}).\text{astype}(\text{int})$: muestreo de Bernoulli — si un número aleatorio $\in [0,1]$ cae por debajo de la probabilidad, se asigna retención = 1.

Paso 2 — Regresión Lineal con Estadísticos t

```
train_data, test_data = train_test_split(df_empleados,
                                         test_size=0.2, random_state=123)

modelo_salario = LinearRegression()
modelo_salario.fit(X_train, y_train)

# Errores estándar y t-estadísticos (sin statsmodels)
X_c = np.column_stack([np.ones(len(X_train)), X_train])
mse_tr = np.sum(residuos**2) / (len(y_train) - X_c.shape[1])
cov_mat = mse_tr * np.linalg.inv(X_c.T @ X_c)
se = np.sqrt(np.diag(cov_mat))
t_stats = coefs / se
```

- `train_test_split(test_size=0.2)`: 80 % para entrenar, 20 % para evaluar. `random_state` garantiza reproducibilidad de la partición.
- `np.linalg.inv(X'X)`: inversión de la matriz de información. La covarianza de los coeficientes es $\text{Cov}(\beta) = \sigma^2 \cdot (X'X)^{-1}$.
- $\text{mse}_{\text{tr}} = \text{SSR}/(n-p)$: error cuadrático medio de los residuos de entrenamiento. p = número de parámetros (intercepto + covariables).
- $t = \beta/\text{SE}(\beta)$: si $|t| > 2$, el coeficiente es significativamente distinto de cero (para n grande, corresponde a $p < 0.05$ aproximadamente).

Métrica	Fórmula	Interpretación
Coeficiente (β)	$\text{solve}(X'X, X'y)$	Cambio en salario por unidad de la variable
R^2 entrenamiento	<code>modelo.score(X_train, y_train)</code>	Ajuste del modelo a los datos de entrenamiento
R^2 prueba	<code>r2_score(y_test, y_pred)</code>	Capacidad de generalización (datos no vistos)
RMSE	$\sqrt{\text{MSE}_{\text{test}}}$	Error promedio de predicción en unidades de \$ del salario
t-estadístico	<code>coef / SE</code>	Significancia estadística de cada predictor

Paso 3 — Clasificación KNN con GridSearchCV

```

scaler_knn = StandardScaler()
X_knn_train_sc = scaler_knn.fit_transform(X_knn_train)
X_knn_test_sc = scaler_knn.transform(X_knn_test)  # mismo scaler, NO re-ajustar

param_grid = {"n_neighbors": range(1, 21, 2)}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_knn = GridSearchCV(KNeighborsClassifier(), param_grid,
                        cv=cv, scoring="accuracy", n_jobs=-1)

```

- `scaler_knn.transform(X_knn_test)`: aplica los parámetros ajustados en entrenamiento al conjunto de prueba. NO se re-ajusta para evitar data leakage.
 - KNN clasifica un empleado asignándole la clase más frecuente entre sus k vecinos más cercanos en el espacio escalado.
 - StratifiedKFold mantiene la proporción de clases (0/1) en cada pliegue: esencial con clases desbalanceadas.
 - `n_jobs=-1`: usa todos los núcleos del procesador disponibles para paralelizar la búsqueda de k óptimo.
 - `range(1, 21, 2)`: k impares {1, 3, 5, ..., 19} — impares evitan empates en la votación KNN.
-
- El gráfico `plt.errorbar` muestra Accuracy $\pm$ SD para cada k: permite elegir el k con mejor precisión y menor varianza.

Paso 4 — Clustering K-Means

```

scaler_km = StandardScaler()
datos_clustering_sc = scaler_km.fit_transform(datos_clustering)

kmeans = KMeans(n_clusters=3, random_state=456, n_init=10)
kmeans.fit(datos_clustering_sc)
df_empleados['cluster'] = kmeans.labels_.astype(str)

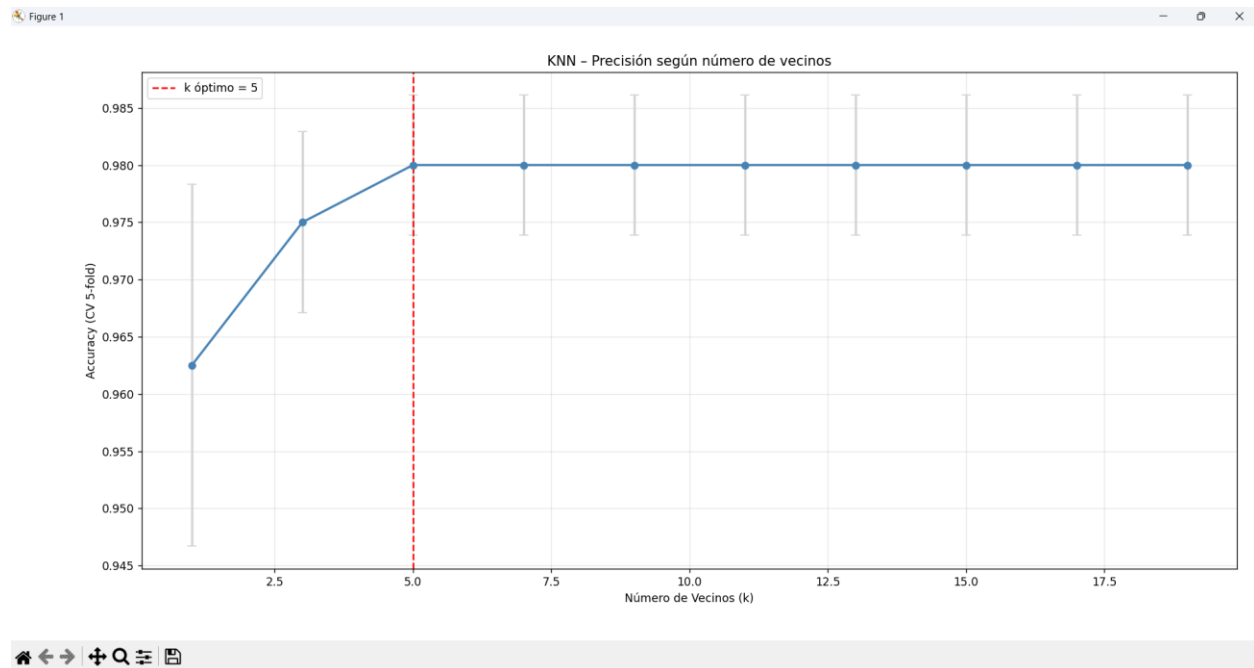
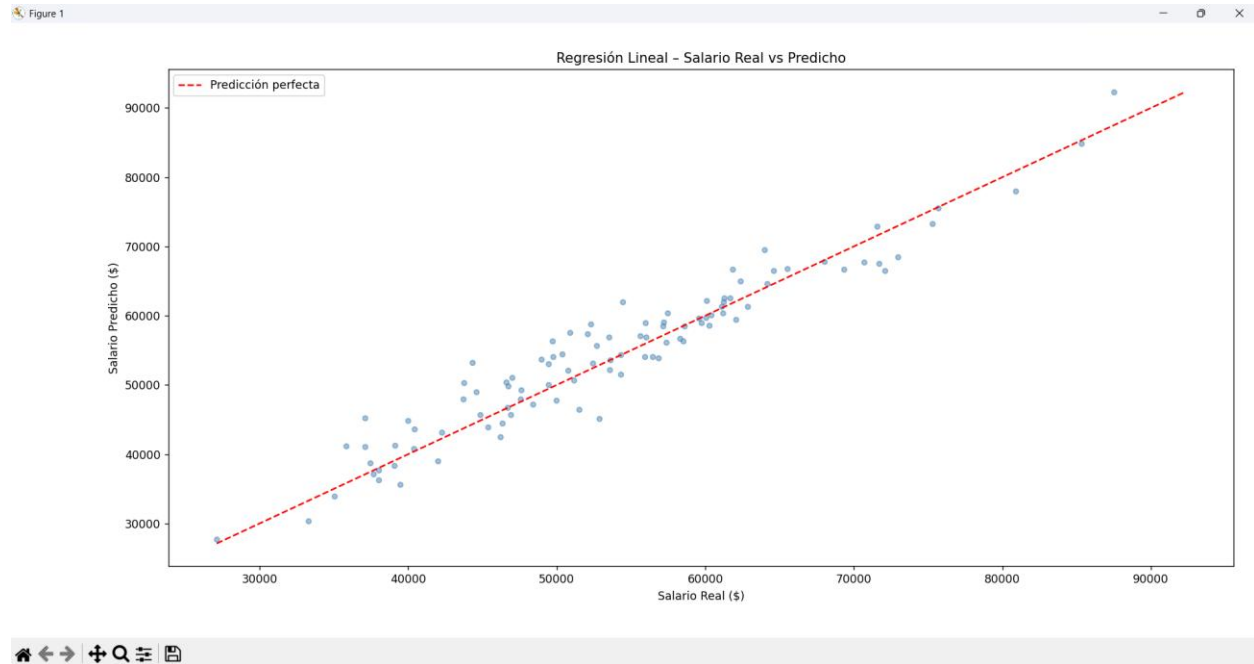
# Etiquetado automático de perfiles
if exp_media < 4 and sal_media < 45000: etiquetas[c] = "Junior"

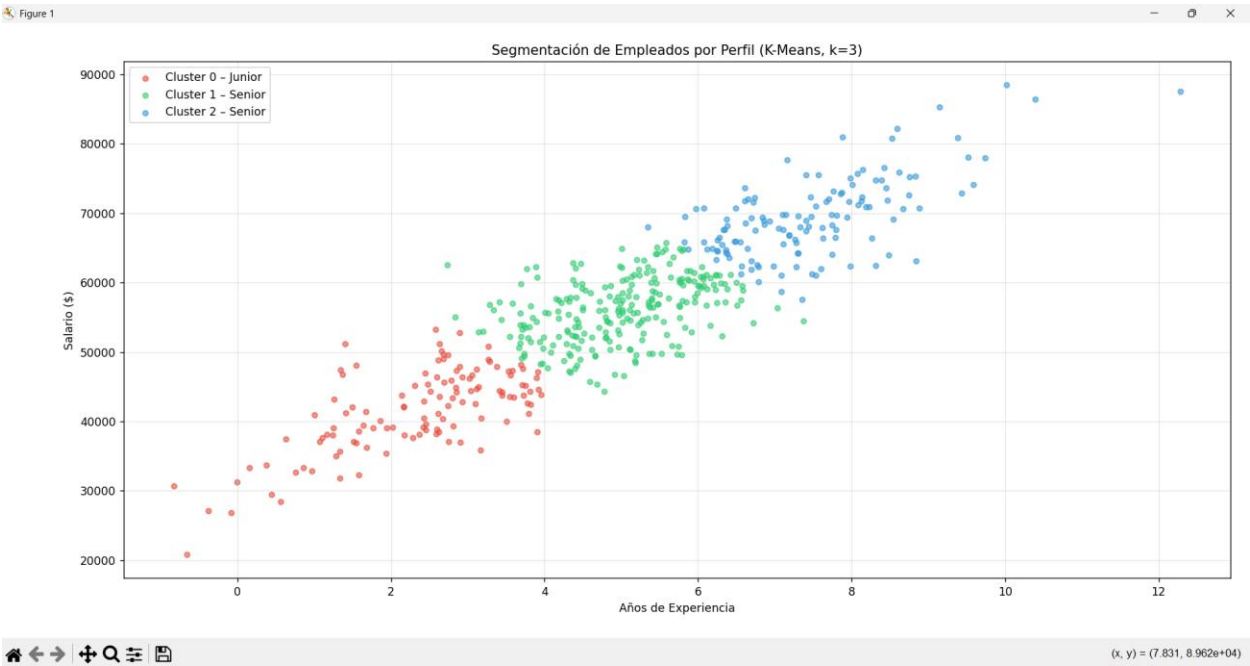
```

```
elif exp_media > 6 or sal_media > 55000: etiquetas[c] = "Senior"
else: etiquetas[c] = "Mid"
```

- `n_clusters=3`: se asume 3 perfiles naturales (Junior, Mid, Senior).
- `n_init=10`: ejecuta el algoritmo 10 veces con centroides iniciales aleatorios distintos y selecciona el resultado con menor inercia (suma de distancias cuadradas al centroide).
- `labels_.astype(str)`: convierte las etiquetas numéricas {0,1,2} a strings para facilitar el manejo en Pandas y la leyenda del gráfico.
- El bloque de etiquetado automático asigna nombres legibles (Junior/Mid/Senior) basándose en la media de experiencia y salario de cada cluster.

Algoritmo	Tipo	Función en la práctica
LinearRegression	Supervisado · Regresión	Predice salario en función de experiencia y certificaciones
KNeighborsClassifier	Supervisado · Clasificación	Predice retención basándose en empleados similares
KMeans	No supervisado · Clustering	Agrupar empleados sin etiquetas: descubre perfiles automáticamente
GridSearchCV	Meta-estimador	Selecciona el k óptimo de KNN por validación cruzada
StandardScaler	Preprocesamiento	Estandariza para KNN (distancias) y K-Means (centroides)





Resumen General: Librerías y Herramientas por Práctica

Librería / Módulo	P2	P3	P4	P5	P6	P7
numpy	✓	✓	✓	✓	✓	✓
pandas	✓	✓	✓	✓	✓	✓
matplotlib	✓	✓	✓	✓	✓	✓
seaborn	✓	✓	✓	—	✓	—
scipy.stats	✓	✓	—	—	—	—
sklearn.PCA	—	—	✓	✓	✓	—
StandardScaler	—	—	✓	✓	✓	✓
LinearRegression	✓	✓	—	—	—	✓
KNeighborsClassifier	—	—	—	—	—	✓
KMeans	—	—	—	—	—	✓
GridSearchCV	—	—	—	—	—	✓

Práctica	Tema Central	Técnicas Clave
P2	Calidad Industrial	EDA, Pearson, Welch t-test, Regresión Lineal, IC 95%

P3	Logística Global	EDA, Pearson, Welch t-test, Cohen's d, Regresión por grupo
P4	PCA Sensores Industriales	PCA, StandardScaler, Scree Plot, Kaiser, Loadings, Biplot
P5	PCA Tráfico de Red	PCA, Redundancia de red, Loadings, Biplot, Doble umbral
P6	PCA Smart Cities	PCA, LoRaWAN, Heatmap de Loadings, Biplot coloreado
P7	ML RRHH	Regresión Lineal, KNN+CV, K-Means, sigmoid, t/z estadísticos